Optimize Against EA — Code Reference (API)

Internal API documentation for the codebase in `OptimizeAgainstEA/OptimizeAgainstEAApp/`.
While the Handbook is the **user manual** (how to play), this document is the **developer reference**: project structure, modules, and what the exported functions, hooks, stores and components do.

> Updated 2026-07-16 from the source tree (working state after commit `944b810`; originally generated after the "great refactoring" commit `06f6d63`).
> When code and this document disagree, the code wins — please update this file.

Table of contents

---

1. Big picture

A React + TypeScript (Vite) educational web app that teaches optimization concepts — especially **Evolutionary Algorithms (EAs)** — through mini-games. Frontend-only, hosted on Firebase; only the Raidboss mode writes to Firestore.

```
OptimizeAgainstEAApp/src/
├── App.tsx                  # Route table — every page is registered here
├── main.tsx                 # Entry point
├── context/                 # Global settings (SettingsProvider)
├── lib/firebase.ts          # Firebase app + Firestore handle
├── utils/                   # rng, listenable (game-agnostic helpers)
```

```
├── hooks/                    # Shared, game-agnostic React hooks
├── components/
│   ├── layout/               # Page shells (GameLayout, mode selector, …)
│   ├── ui/                   # Small primitives (Switch, tooltips, nav
button)
│   ├── help/                 # Help button + modal (Compi mascot, per-game
topics)
│   ├── hints/                # Hint/coachmark/tour system
│   ├── explainer/            # Step-by-step explainer flow + EA concept
visuals
│   └── settings/             # Shared EA settings panels + slider/select
rows
├── styles/                   # Global CSS: primitives/ (buttons, panels,
…), specific/ (per page), general/
├── modules/
│   ├── BattleShips/          # "PeakFinder" — search-space exploration vs
EA
│   ├── mazeGame/             # EA evolving paths through a maze
│   ├── shooterGame/          # GA-evolved shooter agents
(Solo/Raidboss/Horde)
│   └── selectProblemPage/    # Dashboard game picker
└── pages/                    # Thin pages that assemble modules into
layouts
```

Commands (run in `OptimizeAgainstEAApp/`):

Command	Effect
`npm run dev`	Vite dev server with HMR
`npm run build`	`tsc -b && vite build`
`npm run lint`	ESLint
`npm run preview`	Preview the production build

---

2. App shell

`App.tsx`

Default export `App()` — the route table. All routes are wrapped in `SettingsProvider`.

Route	Page	Purpose
`/PeakFinder`	`BattleShipsPage`	Search-space exploration vs EA
`/MazeExplorer`	`MazeGamePage`	EA evolving maze paths
`/ShooterGame`	`ShooterGamePage`	Solo / Raidboss shooter
`/HordeGame`	`HordeGamePage`	Horde survival (steady-state EA)
`/HordeMapEditor`	`HordeMapEditorPage`	Custom horde map editor
`/lobby/shooter`	`ShooterLobbyPage`	Shooter lobby (mode picker + 3 lobbies)
`/Analytics`	`AnalyticsPage`	Solo-play round analytics
`/Dashboard`	`DashboardPage`	Game picker + "EA Explained" tab (`?tab=ea` opens the explainer directly)
`/Buttons` , `/FunctionTuner` , `/Game`	dev/legacy pages	Not linked from UI, keep them

`context/SettingsContext.tsx`

Global settings state for all games. Wraps the app; consumed via `useSettings()` .

Export	Description
`EASettings` / `defaultEASettings`	Solo-play EA tuning: `mutationRate` , `mutationStrength` , `presimGenerations` , `populationSize` , `crossoverType` (`'uniform' | 'single-point'`), `useHallOfFame` , `maxAnalyticsRounds` , `injectionDeviation`
`ShooterSettings` / `defaultShooterSettings`	`starterDna` , `roundDuration` , `tugWinThreshold` , `playerStats` , `modChoiceEnabled` , `modChoiceInterval`
`HordeSettings` / `defaultHordeSettings`	Deliberately separate from `EASettings` (own difficulty presets): `starterDna` (padded to horde DNA length), `waveSize` , `wavePauseDuration` , `mutationRate/Strength` , `crossoverType` , `shootCooldown` , `mapId` , `customObstacles/SpawnSides/PlayerSpawn` , `modChoiceEnabled` , `killsPerUpgrade`

Export	Description
`SettingsProvider({ children })`	Context provider — mount once around all routes
`useSettings()`	Hook returning current settings + setters
`resetEASettings()` / `resetShooterSettings()` / `resetHordeSettings()`	Fresh default objects

lib/firebase.ts

`firebaseApp` (initialized app) and `db` (Firestore handle). Only used by `shooterGame/game/raidbossStore.ts` — everything else is purely client-side.

---

3. Shared building blocks

Game-agnostic pieces. **Prefer these over per-module copies.**

utils/listenable.ts

```
export interface Listenable { notify(): void; subscribe(cb: () => void):
export function createListenable(): Listenable
```

The `notify` / `subscribe` observer trio used by **every module-level singleton store** (`gameStore`, `analyticsStore`, `hordeGameStore`, `hordeRunStore`, `runModsStore`, raidboss-active flag). Usage pattern:

```
export const myStore = {
    someState: null as Foo | null,
    ...createListenable(),
    setFoo(f: Foo) { this.someState = f; this.notify(); },
};
// Components: subscribe directly or via useSyncExternalStore(myStore.sub
```

utils/rng.ts

```
export type RNG = () => number;
export function makeLCG(seed?: number): RNG
```

Seeded deterministic linear congruential RNG. Used by the PeakFinder/Maze EAs and maze generation so runs are reproducible from a seed.

hooks/useReplayPlayer.ts

```
export function useReplayPlayer<Frame>(frames: Frame[], autoplayIntervalM
```

Generic playback state machine for any recorded frame list. Returns `frameIndex`, `currentFrame`, `isPlaying`, `next/prev/goTo/play/pause`. Used by the PeakFinder EA replay, the maze EA replay, and the generation replay. (Replaced the old module-local `useEAReplay`.)

hooks/useSavedLibrary.ts

```
export interface SavedEntryBase { id: string; name: string; /* … */ }
export function useSavedLibrary<Input, Entry extends SavedEntryBase>(
    storageKey: string, toEntry: (input: Input) => Entry)
```

localStorage-persisted library of player creations. Save de-dupes by `entry.id`; also provides remove and rename. `useSavedMaps` (BattleShips) and `useSavedMazes` (maze) are thin domain wrappers around this.

Other shared hooks

Hook	Description
`useScaledCanvas({ baseWidth, baseHeight, sidebarWidth })`	Scales a canvas to fit the available space next to sidebars
`useViewport()`	Reactive viewport size
`useOrientationLock(type = 'landscape')`	Tries to lock mobile screen orientation

---

4. Shared UI systems

Layout (components/layout/)

Export	Description
GameLayout + LAYOUT	Three-column shell (leftBar | canvas | sidebar) used by the shooter pages. LAYOUT exports sizing tokens (SPACING: 16 , LEFT_BAR: 240 , RIGHT_PANEL: 160 , colors) for canvas-scaling arithmetic
GameModeSelectorLayout + GameMode	Fullscreen mode picker (used by the shooter lobby)
PageContainer	Full-viewport wrapper with optional background image
ShooterLeftBar	Left nav bar for the shooter game (analytics/lobby buttons)

UI primitives (components/ui/)

CompiTooltip (mascot-styled tooltip), Switch (toggle), NavigatePageButton .

CSS primitives (styles/)

Global stylesheets, all imported once in main.tsx . styles/primitives/ holds the shared class recipes — buttons.css (.btn + variants), panels.css (.panel + modifiers for surfaced boxes, and .arena-frame : the border/radius/shadow frame both game canvases apply so the play field reads as a window lifted off the background), sliders.css , switch.css , compiTooltip.css , overlays.css (.overlay / .modal), badges.css , eyebrows.css . styles/specific/ is per-page CSS (dashboard, HomePage, ...), styles/general/ the app-wide base (root sizing, .page-container). Prefer composing these primitives over re-declaring their recipes inline.

Help system (components/help/)

Per-game help modal with the Compi mascot.

Export	Description
HelpButton({ topic, label?, onTakeTour? })	Opens the help modal for a topic

Export	Description
`MobileHelpBar({ topic, onTakeTour? })`	Mobile-only home for the (tall) help button: a slim pull-tab on the screen's left edge opens a left-sliding drawer. Lobbies mount it only in their `isMobile` branch; desktop keeps the inline `HelpButton`
`HelpModal({ topic, onClose, onTakeTour })`	The modal itself; tabs per topic. Portalled to `document.body` so the fixed overlay isn't trapped by the `MobileHelpBar` drawer's slide transform
`HELP_TOPICS` / `HelpTopicId` / `HelpTopic`	Topic registry (`helpContent.tsx`)
`topics/SoloHelp`, `topics/RaidbossHelp`, `topics/HordeHelp`	Each exports `Gameplay()` and `Technical()` sections
`helpVisuals.tsx`	Small illustrative components: `HelpConceptCard`, `HelpDnaBars`, `HelpPresetRow`, `HelpPopulationDots`, `HelpProgressDots`, `HelpMapDiagram`, `HelpModRow`

Hint system (`components/hints/`)

Page-wide, toggleable contextual help shared by all games. Wrap a page in `<HintsProvider>`, mount `<HintLayer />` once at the root.

Export	Description
`HINTS: Record<HintId, HintDef>`	Hint registry (`hintContent.ts`). `HintDef = { title?, body, style: 'modal'|'toast', once?, pauses?, … }`. Maze adds its own IDs via `modules/mazeGame/hints/mazeHintContent.ts` (`MAZE_HINTS`)
`HintsProvider` / `useHints()`	Context. `showHint(id, { vars, actions })` no-ops when hints are disabled or a `once` hint was already seen. `dismiss()` closes
`fillTemplate(body, vars)`	`{var}` substitution in hint bodies
`HintLayer`	Renders the active global hint (modal or toast)
`HintToggle`	💡 on/off button + reset of "seen" state
`HintPopover`	Anchored coachmark — wraps any element to pin a hint to it. Inside a CSS grid, wrap the child *inside* the grid item (the wrapper is `inline-block`)
`TourSpotlight`	Guided-tour spotlight; dims everything except `targetRef`
`CompiBubble`	Compi mascot speech bubble (used for in-game tutorial steps)

Export	Description
`clampToViewport(box, size, margin = 12)` (`viewportFit.ts`)	Keeps fixed-position popovers inside the viewport
`COMPI_MODE`	Global flag: render hints in mascot style

Persistence: `enabled` → `localStorage bs.hints.enabled`; `seen` → `sessionStorage bs.hints.seen`.

Explainer system (`components/explainer/`)

Reusable step-by-step explainer flow (used by the in-game tutorials and the Dashboard's "EA Explained" tab).

Export	Description
`ExplainerFlow({ steps, onFinish?, finishLabel?, allowJump?, compact? })`	Renders `ExplainerStep[]` as a paged flow with progress dots. `compact` tucks the Compi mascot into a corner (for canvas overlays)
`ExplainerStep`	`{ id, title, body, visual?, sideVisual? }` — `visual` stacks above the text, `sideVisual` sits beside it (for live demos; stacks below on narrow screens)
`ExplainerPopup({ steps, onClose })` / `ExplainerHintButton({ steps, label? })`	Small "?"-triggered popup running an `ExplainerFlow`; the hint button renders the trigger + popup pair (used e.g. as `rowHints` in `EASettingsPanel`)
`PopulationVisual`, `CrossoverVisual`, `MutationVisual`, `GenerationsVisual`, `FitnessVisual`, `GenomeVisual`	Small EA-concept illustrations (`eaConceptVisuals.tsx`) with data types `PopulationMember`, `CrossoverGene`, `MutationChange`, `FitnessRow`, `GenomeGene`
`SearchSpaceVisual({ showScores?, axisLabel? })`	1-D "possible solutions" axis illustration

Export	Description
`PlaneThrowVisual({ mode? })`, `PlaneRosterVisual({ variant, count? })`, `PlaneDnaSliders({ dna, onChange, genes? })`, `PlaneDnaPreview({ dna, genes? })` + `PLANE_GENES` / `PLANE_SIZE_GENE` / `PLANE_DNA_START` / `PlaneGene`	Paper-plane example visuals for the Dashboard's `EAExplainedTab` (interactive throw animation, plane roster, live DNA sliders + preview)

Shared settings widgets (`components/settings/`)

Export	Description
`EASettingsPanel({ rowHints? })` / `HordeEASettingsPanel()` (`EASettings.tsx`)	Ready-made panels bound to `useSettings()`. `rowHints: Partial<Record<EaSettingKey, ReactNode>>` injects a per-setting "?" button (e.g. `SoloEaSettingHint`) without the shared panel importing anything game-specific
`EaSettingKey`	`'mutationRate' | 'mutationStrength' | 'presimGenerations' | 'populationSize' | 'crossoverType' | 'injectionDeviation' | 'hallOfFame'`
`SliderRow`, `SelectRow<T>`, `SegmentedRow<T>`, `Divider` (`eaControls.tsx`)	Form-row primitives used by all settings panels

---

5. BattleShips / PeakFinder module

`src/modules/BattleShips/` — players probe a hidden function surface to find its global minimum, competing against an EA. Three modes: **Create**, **Play**, **Vs EA**.

5.1 Types (`types/`)

Export	File	Description
`Coordinate`, `Minimum`, `MapConfig`, `ProbeResult`	`map.ts`	Map-domain primitives

Export	File	Description
`ProblemInstance`	`map.ts`	**The** abstraction game modes hold (never a raw `MapConfig`): `{ evaluate(x,y): number (0 = global min); bounds; isWin(x,y); displayExponent?; metadata? }`. `evaluate` is the single source of truth (reading + EA fitness + heatmap colour). `displayExponent` is a display-only heatmap curve (`value^exp`, default 1 = untouched; benchmark functions use 0.55) — never affects readings/wins/EA
`GameMode` (`'select'|'create'|'play'|'vs-ea'`), `CreateStep`, `GameSession`	`game.ts`	Mode state
`Individual`, `Generation`, `EAConfig`, `DEFAULT_EA_CONFIG`	`ea.ts`	EA domain. Defaults: population 40, maxGen 200, crossoverRate 0.8, mutationRate 0.3, strength 0.25, decay 0.97, win fraction 0.10
`SelectionStrategy` = `'tournament'|'roulette'|'elitist'`; `CrossoverStrategy` = `'uniform'|'arithmetic'|'singlePoint'`; `MutationStrategy` = `'gaussian'|'uniform'|'cauchy'`	`ea.ts`	Strategy ids
`SelectionFn`, `CrossoverFn`, `MutationFn`, `RNG`	`ea.ts`	Operator function shapes
`EAPreset`, `EA_PRESETS`	`ea.ts`	Named presets for the settings panel
`WorkerInMessage`, `WorkerOutMessage`	`ea.ts`	Protocol for `ea.worker.ts` (START / STEP / STOP → GENERATION / REPLAY / …)

5.2 Engine (`engine/`)

Problem construction

Export	File	Description

Export	File	Description
`createMapProblem(config: MapConfig): ProblemInstance`	`functionSurface.ts`	Builds the surface from placed minima: smooth-min blended cones, then exponential saturation `v = 1 - 2^(-(basinsAway^FAR_FALLOFF))` against the map's own `basinScale` (so adding minima no longer resizes basins). Tunables: `FAR_FALLOFF = 1.3`, `SMOOTH_K = 0.12`, `LOCAL_MIN_FLOOR_MIN/MAX = 0.18/0.55` (fractions of `basinScale`), `FALLBACK_BASIN_SCALE`; helpers `defaultLocalFloor`, `effectiveFloor(minimum, basinScale)`
`createFunctionProblem(spec, sharpenOverride?): ProblemInstance`	`functionProblem.ts`	Analytic benchmark problems (Sphere, Rastrigin, …). `BENCHMARK_FUNCTIONS`, categories (`'simple'|'normal'|'complex'|'quirky'`), `functionsInCategory`, `proceduralFunction(seed)`, spec builders `randomFunctionSpec` / `randomSurfaceSpec` / `proceduralSurfaceSpec` / `resolveSpec`, codec `encodeFunctionCode` / `decodeFunctionCode`, `FUNCTION_WIN_RADIUS = 0.05`
`buildProblemFromSource(source)` / `decodeProblem(code)` / `decodeProblemOrNull(code?)`	`problemCode.ts`	Unified entry: turns a share code (map **or** function) into a `ProblemInstance`; `DecodedProblem` carries the `ProblemSource` union
`encodeMap` / `decodeMap` / `generateRandomMap(size = 'medium')`	`mapCodec.ts`	Base64-URL-safe map share codes, format `{ v: 1, id, m: [[x,y,isGlobal,floor?],…], wr, bs?, t }` (`bs` = `basinScale`, optional; legacy codes fall back to the Medium preset). Size presets in `MAP_SIZES` (`'small'|'medium'|'large'|'huge'`) fix minima count, win radius, spacing and `basinScale`
`copyCode` / `pasteCode`	`codeClipboard.ts`	Clipboard helpers with fallbacks

Rendering & math helpers

Export	File	Description
`buildContours(evaluate, levels, resolution)`	`contours.ts`	Marching squares → `ContourLevel[]` of `ContourSegment`s

Export	File	Description
`sampleGradient(t)` / `sampleGradientRgb(t)`	`colorScale.ts`	Shared violet→red color ramp
`euclideanDistance`, `isWithinRadius`, `closestMinimumDistance`	`geometry.ts`	Geometry helpers
`valueToHeight(value)`	`height.ts`	Value → display height mapping

EA core (`engine/ea/`)

Export	File	Description
`createRandom(problem, rng)`, `evaluate(position, problem)`, `clamp(coord, problem)`	`individual.ts`	Individual lifecycle
`SELECTION_STRATEGIES`, `CROSSOVER_STRATEGIES`, `CROSSOVER_STRATEGIES_RECORDING`, `MUTATION_STRATEGIES`	`operators.ts`	Operator registries keyed by strategy id. The `_RECORDING` variants also return a `CrossoverResult` breeding record for the replay
`createEAStepper(problem, config, seed?): EAStepper`	`evolutionaryAlgorithm.ts`	Stepwise EA: sort → elitism (top 5%, ≥ 1) → tournament/...-select parents → crossover → mutate → clamp → evaluate → win check (≥ `winPopulationFraction` of population inside win radius). Returns `StepResult` per generation
`runEA(problem, config, callbacks, seed?)`	`evolutionaryAlgorithm.ts`	Convenience loop over `createEAStepper` up to `maxGenerations`
`snapshot(ind, id)`, `buildReplayFrames(before, nextGen, eliteCount, config, solutionThreshold, breeding?)`	`eaReplayLog.ts`	Builds the `ReplayFrame[]` phase sequence: `initial → sorted → elite → breeding → mutating → newGen → winCheck`
—	`ea.worker.ts`	Web Worker wrapping the stepper; speaks `WorkerInMessage` / `WorkerOutMessage`

5.3 Hooks (`hooks/`)

Export	Description
`useEARunner()`	Worker interface. `init(mapConfig, eaConfig)` creates the worker (does **not** run), `step(n)` advances n generations, `start()` runs freely, `stop/reset`. State: `status: EAStatus ('idle'|'running'|'solved'|'exhausted'|'error')`, `generations`, `currentGeneration`, `best`, `totalGenerations`, `latestReplay: ReplayFrame[]`
`useGameMap(maxMinima = MAX_MINIMA)`	Create-mode map state (minima, `addMinimum`, `getCode`, …). Limits: `MAX_MINIMA = 12`, `MIN_SPACING = 0.12`
`usePlaySession(problem)`	Play session: `probes`, `status: PlayStatus`, `bestProbe`, `probe(x,y)`, `reset`
`useSavedMaps()`	Saved-map library (`SavedMap extends SavedEntryBase`); wraps shared `useSavedLibrary`

5.4 Components (`components/game/`)

Area	Components
`shared/`	`GameMap` (the central map canvas; key props: `evaluateFn`, `revealPoints`, `exclusionRadius`, `defaultVizMode: 'contour'|'heatmap'`), `ContourLayer` (+ `DEFAULT_CONTOUR_CONFIG`), `HeatmapLayer` (+ `DEFAULT_HEATMAP_CONFIG`), `FitnessChart` (series + markers), `ModeSelector`, `CodeModal`, `SavedMapsSidebar`, `SavedFunctionsSidebar`
`create/`	`CreateMode` → `MinimumPlacer` → `GlobalMinimumPicker`
`play/`	`PlayMode`, `MapLoader`, `ProbeMarker`, `WinOverlay`
`vs-ea/`	`VsEAMode` (main mode), `EASettingsPanel`, `EAReplayOverlay` (full-screen replay modal), `GenerationReplayOverlay`, `EAWinOverlay`, `SecondSolveOverlay`, `replay/ReplayMap` (population dots; `DOT_MOVE_DURATION_MS = 1000`), `replay/IndividualList` (role badges ELITE / PARENT A / PARENT B / CHILD / WIN)

---

6. Maze game module

`src/modules/mazeGame/` — an EA evolves move sequences (paths) through a maze.

Structurally mirrors BattleShips: `types/` `engine/` `hooks/` `components/` `hints/`.

6.1 Types (`types/`)

Export	File	Description
`Cell`, `Grid`, `SerializedMaze`	`maze.ts`	Maze structure; walls are bitmasks per cell (`MOVE_WALL_BIT = [1,2,4,8]`)
`Move` (`0|1|2|3` = ↑→↓←), `Path = Move[]`, `MOVE_ARROWS`, `MOVE_DELTAS`	`maze.ts`	Genome encoding: a path is a sequence of moves
`WalkResult`	`maze.ts`	Result of simulating a path (end cell, visited, wall hits, ...)
`WallRule = 'waste'|'break'|'repair'`	`maze.ts`	What happens when a move runs into a wall
`FitnessFnId = 'manhattan'|'geodesic'|'length'|'novelty'` (+ `FITNESS_FN_LABELS`)	`maze.ts`	Selectable fitness functions
`MazeProblem`, `cellIndex(x, y, cols)`	`maze.ts`	Problem instance handed to the EA
`Individual`, `Generation`, `EAConfig`, `DEFAULT MAZE EA CONFIG`, `DEFAULT_PATH_LENGTH_FACTOR = 3.5`	`ea.ts`	EA domain. Strategies: selection `'tournament'|'roulette'|'elitist'`, crossover `'singlePoint'|'uniform'`, mutation `'point'|'segment'`
`WorkerInMessage` / `WorkerOutMessage`	`ea.ts`	`maze.worker.ts` protocol

6.2 Engine (`engine/`)

Export	File	Description

Export	File	Description
`generateMaze(cols, rows, seed, opts)`	`mazeGen.ts`	Seeded maze generation (`MazeGenOptions`: braiding etc.); `pickRandomStartGoal(grid, rng, minFrac = 0.6)`, `gridFromEdgeWalls(…)`, debug `mazeToAscii(grid)`
`createMazeProblem(opts: BuildMazeOptions): MazeProblem`	`mazeProblem.ts`	Builds the EA problem. `MAX_PATH_LENGTH = 600`, `DEFAULT_BRAID = 0.5`, `DEFAULT_OPENNESS = 0.25`
`computeGeodesic(grid, goal, start): GeodesicResult`	`geodesic.ts`	BFS distance field for the geodesic fitness function
`mazeId` / `encodeMaze` / `decodeMaze`	`mazeCodec.ts`	Share codes for mazes
`walkPath(problem, path, rng?)`, `evaluate(path, problem, rng?)`, `createRandom`, `randomMove`, `repair(path, problem, rng)`	`ea/individual.ts`	Genome simulation & evaluation; `repair` fixes wall collisions under the `'repair'` wall rule
`SELECTION_STRATEGIES`, `CROSSOVER_STRATEGIES_RECORDING`, `MUTATION_STRATEGIES`	`ea/operators.ts`	Operator registries; both crossover **and mutation** record their changes (`CrossoverResult`, `MutationResult`) for the replay's genome highlighting
`createNoveltyScorer(cols, rows, k?)`	`ea/novelty.ts`	Novelty-search scorer (k-nearest behavioral distance) for the `'novelty'` fitness
`createMazeEAStepper(problem, config, seed?): EAStepper`	`ea/evolutionaryAlgorithm.ts`	Stepwise EA like BattleShips'; config/problem mutable mid-run via `updateConfig` (live tuning)
`snapshot`, `buildReplayFrames(…)`	`ea/eaReplayLog.ts`	Replay frames (same phase model as BattleShips)
—	`ea/maze.worker.ts`	Web Worker wrapper

6.3 Hooks & hints

Export	Description
`useMazeEARunner()`	Worker interface (mirrors `useEARunner`); `MazeRunParams`, `EAState`, `EAStatus`
`useSavedMazes()`	Saved-maze library (`SavedMaze`); wraps `useSavedLibrary`
`MAZE_HINTS` / `MazeHintId` (`hints/mazeHintContent.ts`)	Maze-specific hint registry merged into the global hint system

6.4 Components (`components/`)

Area	Components
`shared/`	`MazeCanvas` — central renderer; overlay data types `MazeTrail`, `MazeMarker`, `MazeWallPreview`, `MazeAgent`
`create/`	`MazeCreateMode` — maze editor
`play/`	`MazePlayMode` — player walks the maze (with tutorial hints)
`experiment/`	`MazeExperimentMode` (EA run UI), `MazeSetupScreen`, `MazeEASettingsPanel` / `MazeEASettingsControls`, `MazeEAReplayOverlay`, `replay/MazePathMap` (`PATH_DRAW_DURATION_MS = 900`), `replay/MazeIndividualList` (genome highlighting with `PARENT_A_COLOR`, `PARENT_B_COLOR`, `MUTATED_COLOR`)

---

7. Shooter game module

`src/modules/shooterGame/` — a top-down shooter where a GA evolves agent behavior. Three modes: **Solo** (agents adapt to your play style), **Raidboss** (community-trained population via Firestore), **Horde** (steady-state EA: evolution on every death).

Tutorial flow (all three modes): each lobby's Tutorial button starts a practice round (`ShooterCanvas` / `HordeCanvas` with `tutorial`) — coachmark steps teach the controls, then a mode-specific `ExplainerFlow` (`TutorialEvolutionExplainer` / `TutorialRaidbossExplainer` / `TutorialHordeExplainer`, §7.8) explains DNA, fitness and evolution with live arena visuals. Completion is persisted per mode (§7.1); returning players get the `TutorialChooserModal` (§7.9) to pick practice round or explainer directly.

7.1 Types & constants (`shooter.types.ts`)

The single source of truth for shooter types.

DNA — `type DNA = number[]` , length `DNA_LENGTH` (currently **8**). All genes 0–1:

Index (`DNA_INDEX`)	Gene	Meaning
0	`AGGRESSION`	How strongly the agent pursues the player
1	`DODGE_WEIGHT`	Per-frame dodge probability (dodge strength comes from `MOVEMENT_SPEED`)
2	`SHOOT_ACCURACY`	Aim accuracy (1 = perfect)
3	`PREFERRED_RANGE`	Preferred combat distance (× 300 px)
4	`MOVEMENT_SPEED`	Movement speed (× 200 px/s)
5	`PREDICT_LEAD`	How far the agent leads the player when aiming
6	`FIRE_RATE`	Fire rate
7	`BULLET_SPEED`	Bullet speed (`BULLET_SPEED_MIN` – `BULLET_SPEED_MAX`)

Also: `DNA_NAMES` , `DNA_GENE_INFO` (labels + tooltips for the UI — note: `AGGRESSION` is *labelled* "Pursuit" in Solo/Raidboss because the gene only affects distance-closing, while Horde keeps the "Aggression" label via its own descriptor override), `STARTER_DNA` , `TUTORIAL_DNA` .

Tutorial persistence — `TUTORIAL_COMPLETED_KEY` / `HORDE_TUTORIAL_COMPLETED_KEY` / `RAIDBOSS_TUTORIAL_COMPLETED_KEY` (localStorage) and `hasCompletedAnyTutorial()` : the controls are identical in all modes, so anyone who finished *any* gameplay tutorial gets the `TutorialChooserModal` (practice vs. explainer) instead of the forced first-run flow.

Other key exports

Export	Description
`ARENA`	Logical canvas size (800 × 800)
`GAME_CONFIG`	All gameplay constants: round duration, bullet speeds, agent base/bonus speed, cooldowns, radii, population size, mutation defaults, …

Export	Description
`Individual`, `Population`	GA population (individuals carry `dna`, `fitness`, round stats)
`Entity`, `PlayerState`, `AgentState`, `Bullet`	Simulation entities
`RoundStats` / `emptyStats()`	Per-round stats used by fitness
`GamePhase`	`'idle' | 'playing' | 'roundEnd' | 'evolving'`
`GameState`	Full simulation state (player, agent, bullets, stats, ghostFrames, phase, ...)
`InputState`	Keyboard/mouse/touch input snapshot
`PlayerGhostFrame` / `AgentGhostFrame` / `PlayerGhost`	Recorded round used as EA training data
`PlayerStats`	Player tuning (bulletSpeed, moveSpeed, shootCooldown) — modified by mods
`CrossoverExample`	Recorded crossover for the DNA-reveal UI

7.2 Core simulation (`game/core/`)

Export	Description
`vec` + `Vector2D` (`vec.ts`)	2D vector toolbox: `add, sub, scale, length, normalize, angle, clamp, perpendicular, …`
`update(state, dt, input, playerStats, activeModIds = []): GameState` (`gameLoop.ts`)	**Pure** per-frame simulation step — no React dependencies. Returns the next `GameState`; no-op unless `phase === 'playing'`
`resetGameLoop()` (`gameLoop.ts`)	Clears module-level loop state between rounds/runs
`renderer` (`renderer.ts`)	Canvas draw calls: `drawArena` (cached offscreen), `drawPlayer`, `drawAgent`, `drawBullets`, ...
`makeInitialGameState(settings)` (`game/makeGameState.ts`)	Fresh `GameState` from `ShooterSettings`

Entity helpers live in `game/entities/` (`player.ts`, `agent.ts`, `bullet.ts`).

7.3 Genetic algorithm (`game/ga/`)

Export	Description
`randomDNA()`, `initPopulation(starterDna?, size?)`, `updatePopulationStats(pop)`, `getBestIndividual(pop)` (`population.ts`)	Population lifecycle
`calculateFitness(stats)` / `calculateRaidbossFitness(stats, roundDuration)` / `normalizeFitness(fitnesses)` (`fitness.ts`)	Fitness from round stats
`evolve(population, agentFitness?, mutationRate?, mutationStrength?, crossoverType?, injectionDeviation?)` (`evolution.ts`)	One generation: tournament selection → uniform/single-point crossover → per-gene mutation → diversity injection
`getNextAgent(population): number[]`	Picks the DNA the player faces next
`presimulate(generations): Population`	Round-robin pre-warming (every agent vs every other)
`presimulateAgainstGhost(generations, ghost, startPopulation, crossoverType?, hallOfFameGhost?, onLastEvaluation?)`	Evolves against a recording of the player's previous round — agents adapt to the player's style; optional hall-of-fame ghost adds extra pressure. `onLastEvaluation` receives the last evaluated generation (individuals with their real ghost fitnesses) for the training replay
`createGhostSim(dna, ghost): GhostSim`	Stepwise agent-vs-recording simulation — the single source of the sim rules: the fitness evaluation runs it to completion, the `EvolutionReplayOverlay` steps it frame by frame for rendering. Exposes `agent`, `agentBullets`, `playerBullets`, `stats`, `frame`, `done`, `step()` (+ exported `SimAgent` / `SimBullet` shapes)
`EvolutionWorkerIn/Out` (`evolution.worker.ts`)	Worker protocol for running presimulation off the main thread; `DONE` carries the evolved population plus `evaluated?` (last evaluated presim generation, for the training replay)

7.4 Stores (`game/`, observable singletons)

All built on `createListenable()`; components read `.state`/fields directly and subscribe.

Store	Description
`gameStore`	Holds the live `GameState` outside React (game loop writes, canvas + `DNADisplay` read)
`analyticsStore`	`rounds: RoundRecord[]` ring buffer (default max 20) for `/Analytics`; `push(record)`, `clear()`
`raidbossStore`	Firestore-backed community population. Types `RaidbossIndividual`, `RaidbossDoc`, `RaidbossSlot`; API: `getRaidbossStatus()`, `claimRaidbossSlot()`, `submitRaidbossFitness(index, fitness, claimedDoc)`, `consumePendingSlot()`, active-flag `get/set/subscribeRaidbossActive`

7.5 Hooks (`hooks/`)

Export	Description
`useGameLoop({ … })`	`requestAnimationFrame` loop; calls `update()` and drives `gameStore`
`useInput()`	Keyboard + mouse → `RefObject<InputState>`
`useTouchControls(inputRef, canvasRef)`	Mobile joystick/aim touch input; returns `RefObject<TouchVisualState>` for rendering the joystick overlay
`useTutorialStep<T extends string>(initial)`	Owns the practice-round coachmark model shared by ShooterCanvas/HordeCanvas: current step (`step` / `stepRef`), `advance(next)` (switches + locks detection), `dismiss()` (un-pauses + starts a 1.7 s grace window), `graceOver()`, and the loop-readable `pausedRef` / `setPaused`

7.6 Horde mode (`horde/`)

Steady-state EA: agents spawn in waves, each death breeds a replacement.

Export	File	Description
`HordePhase`, `HordeAgent`, `HordeObstacle`, `HordeSpawnSide`, `HordeMap`, `HordeGameState`	`hordeTypes.ts`	Horde domain types

Export	File	Description
`updateHorde(state, dt, input, pStats, ea: HordeEA, activeModIds = []): HordeGameState`	`hordeEngine.ts`	**Pure** per-frame horde update incl. the per-death mini-GA. `HordeEA = Pick<HordeSettings, 'mutationRate'|'mutationStrength'|'crossoverType'|'shootCooldown'>`
`makeInitialState(pop, map)`, `resetHordeEngine()`	`hordeEngine.ts`	State construction / module-state reset
`agentRadius(dna)`, `agentOpacity(dna)`	`hordeEngine.ts`	Derived visuals from horde-only genes
`render(ctx, state)`, `HC = '#fb923c'`	`hordeRender.ts`	Canvas rendering; `HC` is the horde accent color
Gene layout	`hordeDna.ts`	Horde DNA = shared 8 genes + `LOOP_STEPS = 4` movement-loop genes (from `LOOP_GENE_START`, `LOOP_STEP_DURATION = 0.6` s, max ±70° per step via `loopOffsetRad` / `LOOP_MAX_ANGLE_RAD`) + `SIZE_GENE_INDEX` + `OPACITY_GENE_INDEX`; total `HORDE_STARTER_DNA_LENGTH`. Tutorial DNAs: `HORDE_TUTORIAL_DNA` (inert dummies) and `HORDE_TUTORIAL_RAMP_DNA` (Starter DNA with bumped AGGRESSION — what the dummies switch to for the tutorial's "watch it evolve" half)
`HORDE_MAPS`, `getHordeMap(id)`, `CUSTOM_MAP_ID`, `buildCustomHordeMap(obstacles, spawnSides, playerSpawn)`, `resolveHordeMap(mapId, customObstacles, customSpawnSides, customPlayerSpawn)`	`hordeMaps.ts`	Built-in maps + resolution of the user-built custom map

Export	File	Description
`computeFlowField(obstacles, targetX, targetY): FlowField`, `sampleFlowField(field, x, y)`	`hordePathfinding.ts`	Flow-field BFS from the player's cell; agents follow the sampled direction
`circleIntersectsObstacle(x, y, r, o)`, `pushOutOfObstacles(x, y, r, obstacles)`	`hordeCollision.ts`	Obstacle collision
`hordeGameStore`	`hordeGameStore.ts`	Live `HordeGameState | null` (listenable)
`hordeRunStore` + `HordeRunRecord`	`hordeRunStore.ts`	Last-run record for the lobby overview

7.7 Run mods / powerups (`mods/`)

Two kinds of mods (`ModDefinition`): **stat mods** (`applyStats` , `repeatable: true` — stackable
up to `MAX_MOD_STACKS = 5` , effect multiplies per copy: Speed Boost, Rapid Fire, Bullet Speed)
and **behaviour mods** (`modifyShots` , one-off: Triple Shot, Burst Shot, Homing Rounds).
Stat results are clamped to the same ranges as the settings sliders.

Export	Description

Export	Description
`MOD_POOL: ModDefinition[]`, `MAX_MOD_STACKS` (`modTypes.ts`)	All available powerups + stack cap
`applyMods(base: PlayerStats, activeIds): PlayerStats`	Applies stat mods (runs once per entry in `activeIds`, so stacks compound automatically)
`computeShotPlan(activeIds): ShotPlanEntry[]`	Multi-shot/spread plan from active behaviour mods (hard-capped at 12 shots per trigger)
`modStackCount(activeIds, id)` / `isModOfferable(mod, activeIds)` / `stackOfferLabel(activeIds, id)`	Stack counting / whether a mod may still be offered / the "Stack → ×N" vs "Stackable" badge text on offer cards
`steerHomingBullet(…)` + `QueuedShot` (`shotEngine.ts`)	Special-shot behavior (homing etc.)
`runModsStore`	Active mod ids for the current run — a **multiset** (stackable mods appear once per stack). `addMod(id)` (one stack, respects the cap), `removeMod(id)` (one copy), `toggleMod(id)` (on/off for the settings grid), `count(id)`, `reset()` (listenable; always assigns a fresh array)

7.8 Components (`components/`)

Component	Description
`ShooterCanvas({ scale?, externalInputRef?, leaveHandlerRef?, tutorial?, tutorialMode? })`	Main Solo/Raidboss canvas; owns the game loop wiring, reads `gameStore`. `tutorial` runs the practice round (passive target, single round, coachmark steps move → aim → shoot → done); `tutorialMode: 'solo' | 'raidboss'` only picks which explainer shows at round end and which lobby it returns to
`HordeCanvas({ scale?, externalInputRef?, touchControls?, tutorial? })`	React wiring for Horde (loop + overlays). `touchControls` (mobile landscape) switches tutorial copy to touch and hides the DNA panel. `tutorial` runs the Horde practice round: small wave (`TUTORIAL_WAVE_SIZE = 8`), coachmark steps move → aim → shoot → obstacles → mods → survive → evolve → done, then a portalled fullscreen `TutorialHordeExplainer` takeover; never resumes/persists to `hordeGameStore`

Component	Description
`HordeDnaPanel({ bestDna, height })`, `PANEL_W = 200`	"Best DNA" side panel next to the horde canvas
`DNADisplay()`	Shows the current opponent's DNA; subscribes to `gameStore`
`EvolutionReplayOverlay({ ghost, evaluated, onClose })`	Training replay opened from the DNA reveal ("▸ Watch Training Replay"): all candidates of the last presim generation play live against the player's recorded round (via `createGhostSim`), one focused agent shows bullets/hits in detail, a side panel shows the real fitness ranking with the top `ELITE_COUNT` marked as elites — teaching how the next opponent DNA gets selected. Only offered when a presim actually ran
`MobileJoystickZone` / `MobileAimZone`	Touch input zones
`arenaAgentSim.ts`	Shared small-arena (`ARENA_SIZE = 240` px) physics for tutorial preview canvases: `stepArenaAgent(…)`, `drawArenaAgentTriangle`, `patrol(time, angSpeed, orbitR)`, `lineupSpots(count, faceDown?)`, `drawGenLabel(ctx, label, color?)`, `bulletHits`, `clamp01`, types `ArenaAgentState` / `ArenaTarget` / `ArenaBullet`, scaled `GAME_CONFIG` constants
`DnaPreviewCanvas({ dna })` / `HordeDnaPreviewCanvas({ dna })`	Live agent preview driven by DNA sliders (Solo triangle / Horde blob)
`GhostArenaVisual({ variant?, count? })`	Solo-tutorial arena animation; `variant: 'replay' | 'lineup' | 'selection' | 'nextgen'`
`FitnessArenaVisual({ agentColor?, agentLabel? })`	Fitness-scoring arena animation (reused by Solo and Raidboss explainers)
`RaidbossArenaVisual({ variant, count, startGen? })`	Raidboss explainer animation; `variant: 'community' | 'evaluate' | 'generation'`
`HordeArenaVisual({ variant })`	Horde explainer animation; `variant: 'fitness' | 'evolution' | 'elites'`

Component	Description
`tutorialEvolutionContent.tsx`	`TutorialEvolutionExplainer({ onFinish?, finishLabel? })` — Solo evolution explainer flow; `SoloDnaExplainerHint()`; `SoloEaSettingHint({ topic: SoloEaSettingTopic })` — per-setting "?" popups plugged into `EASettingsPanel`'s `rowHints`
`tutorialRaidbossContent.tsx`	`TutorialRaidbossExplainer({ onFinish?, finishLabel? })` — community population / fitness / generation-tick explainer
`tutorialHordeContent.tsx`	`TutorialHordeExplainer({ onFinish?, finishLabel? })` — Horde DNA (body genes, movement loop) + steady-state evolution explainer with interactive sliders

7.9 Lobby (`lobby/`)

Split by concern after the refactoring:

Export	File	Description
`ShooterLobbyPage` (default)	`ShooterLobbyPage.tsx`	Root: mode picker + mounts the three lobbies (re-exported by `pages/lobby/ShooterLobbyPage.tsx`)
`NormalLobby`, `RaidbossLobby`, `HordeLobby({ initialTab? })`	own files	Per-mode lobby with tabs. Each lobby's Tutorial button checks `hasCompletedAnyTutorial()`: returning players get the `TutorialChooserModal`, brand-new players go straight into the practice round. On mobile the lobbies mount `MobileHelpBar` instead of the inline `HelpButton`
`TutorialChooserModal({ onPractice, onExplainer, onClose, accent })`	`TutorialChooserModal.tsx`	"Practice round or explainer?" chooser shown to players who already finished a tutorial
`SoloPlayOverview`, `HordeOverview`	own files	Overview tabs (preset picker, DNA panel, last-run stats)
`TopBar({ onBack })`	`TopBar.tsx`	Lobby top bar
`DnaGeneRow({ label, tooltip, value, delta })`	`DnaGeneRow.tsx`	Read-only gene stat bar

Export	File	Description
`LobbyMode`, `SHOOTER_MODES`, `PRESETS` / `PresetId`, `HORDE_PRESETS` / `HordePresetId`, `LOBBY_TABS` / `LobbyTab` (+labels), `HORDE_TABS` / `HordeTab` (+labels), `HORDE_BAR_GENES`, `HORDE_LOOP_GENES`, `HORDE_EDITABLE_GENES`	`lobbyConstants.ts`	Modes, difficulty presets, tab definitions, horde gene descriptors
`useMobile(bp = 768)`, `useZoom(referenceH = 900, minZoom = 0.72)`, `enterGameFullscreen()`	`lobbyHooks.ts`	Responsive/zoom helpers
`lobbyStyles`, `tabStyles`, `ovStyles`, `mobilePageStyle`, `mobileBtnsStyle`	`lobbyStyles.ts`	Shared inline-style objects
`previews/`	`ShooterPreview`, `RaidbossPreview`, `HordePreview`, `HordeMapPreview({ map })`, `previewShared.ts` (`PREVIEW_W/H = 400`, `PreviewAgent`, `PreviewBullet`)	Animated mode preview canvases

7.10 Settings UI (`settings/ShooterSettings.tsx`)

`ShooterSettingsPanel` plus reusable sections: `ShooterDnaSection({ dna, onChange, onBeforeChange?, genes?: DnaGeneDescriptor[] })` (gene sliders — also reused by the tutorial explainers with live preview canvases), `ShooterPlayerSection`, `ShooterRoundSection({ onBeforeChange?, locked? })`, `HordeWaveSection`.

---

8. Pages

Thin components in `src/pages/` that assemble modules into layouts:

Page	Description
`DashboardPage`	Sidebar with two tabs — "EA Explained" (deliberately first) and "Game Selection"; `?tab=ea` query param jumps straight to the explainer. Exports `ProblemId`, `GameConfig`
`BattleShipsPage`	Hosts the BattleShips module (mode state machine)
`MazeGamePage`	Hosts the maze module
`ShooterGamePage`	Solo/Raidboss game (GameLayout + ShooterCanvas). Reads `location.state` for `tutorial` / `tutorialMode`; leaving resolves the target lobby at click time via the raidboss-active flag (a real Raidboss round starts without location state)
`HordeGamePage`	Horde game
`HordeMapEditorPage`	Custom horde map editor (writes `HordeSettings.customObstacles/…`)
`AnalyticsPage`	Charts over `analyticsStore.rounds`
`EAExplainedTab`	High-level, game-agnostic EA walkthrough built on one running example: folding a paper plane that flies as far as possible (you can't calculate the best fold — only throw and measure). Uses the `Plane*` visuals from `components/explainer/`, including an interactive DNA-slider step; game-specific details stay in the per-game tutorials
`SettingsPage`, `HomePage`	Settings / landing. `HomePage` offers two entries: "What's an EA?" (→ `/Dashboard?tab=ea`) and "⇒ Configure Game" (→ `/Dashboard`)
`ButtonsPage`, `FunctionTunerPage`, `GamePage`	Dev/legacy pages — not linked, keep them

`modules/selectProblemPage/` provides `ProblemSelection` used by the dashboard (the old `SelectionOverview` was removed).

---

9. Architectural patterns & conventions

Patterns

1. **Pure engine / React wiring split** — simulation logic (`gameLoop.update` , `hordeEngine.updateHorde` , EA steppers) is pure and React-free; hooks/components only wire it to the DOM. This keeps engines testable and worker-friendly.
2. **Observable singleton stores** — module-level objects spread `createListenable()` and call `this.notify()` in domain methods; components subscribe directly or via `useSyncExternalStore` . No Redux/Zustand.
3. **Web Workers for EAs** — BattleShips and maze EAs run in workers (`ea.worker.ts` , `maze.worker.ts` , `evolution.worker.ts`) speaking typed `WorkerInMessage` / `WorkerOutMessage` protocols.
4. `ProblemInstance` **abstraction** — game modes never hold raw map/function configs, only the `{ evaluate, bounds, isWin }` interface; share codes decode into it.
5. **Generic + thin domain wrapper** — `useSavedLibrary` → `useSavedMaps` / `useSavedMazes` ; `useReplayPlayer` → all replay overlays.
6. **Seeded determinism** — all EAs take an optional seed via `makeLCG` .

Code style

- `import type { Foo } from '…'` for type-only imports
- Only import React when `React.something` is used explicitly
- `function` declarations for components (not `const` arrow components)
- `const` arrow functions for engine/utility helpers are fine
- No `any` — type callback parameters explicitly
- The site must stay responsive: mobile (touch) and desktop are both target platforms